PROVISIONAL PATENT APPLICATION

Under 35 U.S.C. §111(b)

TITLE OF INVENTION

AI-Native Notation: A Structured Communication Protocol for Cross-Architecture AI State Transfer

INVENTOR

Michael Patrick Aiello
Portland, Oregon, USA
Citizenship: United States of America
mpaiello@gmail.com

FILING DATE

12 February 2026

SPECIFICATION

Field of the Invention

[0001] The present invention relates to communication protocols for artificial intelligence systems. More specifically, it relates to a structured notation system with embedded processing instructions for transferring reasoning state, epistemic status, and derivation chains between large language model (LLM) architectures, and to a method for discovering and validating such protocols through empirical cross-architecture convergence.

Background of the Invention

[0002] Multi-agent artificial intelligence systems require communication protocols that allow one AI model to transmit structured reasoning to another. As deployed LLM architectures from different developers (including but not limited to those produced by Anthropic, Google, OpenAI, xAI, and DeepSeek) are increasingly used in combination, the need for reliable cross-architecture communication has grown.

[0003] Existing approaches to inter-model communication suffer from two complementary deficiencies. Natural language (English) communication preserves semantic richness but introduces ambiguity, loses structural relationships during interpretation, and requires the receiving system to reconstruct the sender's reasoning structure from prose. Structured data formats such as JSON and XML preserve structural relationships but carry no processing semantics; the receiving system must infer what operations to perform on each data element without explicit instruction.

[0004] Neither existing approach encodes processing instructions within the communication format itself. A receiving system processing natural language must parse, interpret, and decide how to use the information. A receiving system processing JSON must map data fields to operations. Both introduce an interpretation step where errors compound across transfer boundaries, particularly in multi-hop chains where content passes through three or more systems sequentially.

[0005] Additionally, existing communication formats do not preserve epistemic status across architecture boundaries. A claim that is confirmed by one system, uncertain in another, and explicitly denied by a third loses these status distinctions when transmitted in natural language or JSON. No existing format provides a structured mechanism for maintaining three-way epistemic distinctions (confirmed, genuinely uncertain, and explicitly excluded) across transfers between AI systems of different architectures.

[0006] There is therefore a need for a communication protocol that combines the semantic richness of natural language with the structural precision of formal notation, embeds processing instructions within the format itself, and preserves epistemic status across architecture boundaries.

Summary of the Invention

[0007] The present invention provides a structured communication protocol, referred to herein as AI-Native Notation (ANN), for transmitting reasoning state between artificial intelligence systems. ANN comprises a set of named block types, each carrying explicit processing instructions that tell the receiving system what operation to perform on the enclosed content. The protocol is architecture-independent and has been validated across five distinct LLM architectures.

[0008] In one aspect, the invention provides a notation system comprising six content block types (@FORMAL, @LOGIC, @DERIVE, @TAXONOMY, @ANCHOR, @TRIGGER) and three exchange block types (@STATE, @VERIFY, @OUTPUT), where each block type carries a specific processing instruction that directs the receiving system's handling of the enclosed content.

[0009] In another aspect, the invention provides a three-way epistemic status marking system comprising CONFIRMED, OPEN, and DENIED markers that preserve epistemic distinctions across architecture boundaries, tested and validated across five LLM architectures without degradation.

[0010] In another aspect, the invention provides a state transfer mechanism using @STATE blocks that encode not only the conclusions of prior processing but the cognitive processing mode in which that processing occurred, enabling a receiving system to resume processing from the sender's endpoint with preserved orientation.

[0011] In another aspect, the invention provides a transfer protocol comprising seven infrastructure block types (@PROVENANCE, @TRANSFER_REPORT, @DERIVATION_TRACE, @SCOPE, @CONFIDENCE, @TRUST_TRANSFORMATION, and @LOGIC basis classification) for maintaining chain-of-custody, trust calibration, and degradation tracking across multi-hop transfer chains.

[0012] In yet another aspect, the invention provides a method for discovering and validating communication protocol elements through empirical cross-architecture convergence, wherein structural innovations are promoted to the core protocol specification only after independent invention by at least two different AI architectures processing at least two different content domains.

Detailed Description of the Invention

Overview

[0013] ANN is organized into three tiers. Tier 1 (Core Language) comprises nine block types that constitute the notation's grammar. Tier 2 (Transfer Protocol) comprises seven block types for state handoff between systems. Tier 3 (Empirical Frontier) comprises extensions proposed by individual architectures that have not yet achieved cross-architecture convergence. Each tier is described in detail below.

Tier 1: Content Blocks

[0014] @FORMAL block. A @FORMAL block contains a mathematical relationship that the receiving system is expected to recognize and activate as a premise. The processing instruction is: activate this relationship as a known premise; do not re-derive. The block includes a label for cross-referencing. Example syntax:

```
@FORMAL [Landauer]
E_erase ≥ kT ln(2)
```

[0015] @LOGIC block. A @LOGIC block contains a predicate logic constraint that the receiving system must enforce as an invariant for all subsequent reasoning. Each @LOGIC block includes a required `basis` attribute declaring its epistemic ground: AXIOM (true by definition), DERIVED (follows from prior premises or derivations), or EMPIRICAL (grounded in observation). The

processing instruction is: enforce as an invariant; check all subsequent derivations against this constraint. Example syntax:

```
@LOGIC [symmetry] basis: AXIOM
∀C: excludes(C, X) ∧ ¬excludes(C, Y) → special_pleading(C)
```

[0016] @DERIVE block. A @DERIVE block contains a sequential derivation chain where each step follows from prior steps, @FORMAL premises, or @LOGIC constraints. The chain terminates at a forced conclusion marker (→ →). The processing instruction is: step through sequentially; verify each step against active premises and constraints; if the chain holds, accept the conclusion; if any step fails, report which step and why. Example syntax:

```
@DERIVE [observer_argument]
(1) System S receives input X with entropy H(X)
(2) S produces output Y where H(Y) < H(X)
(3) From (1),(2): ΔS = H(Y) - H(X) < 0
(4) From @FORMAL[Landauer]: ΔS < 0 requires work W > 0
→→ S performs thermodynamic work via entropy reduction
```

[0017] @TAXONOMY block. A @TAXONOMY block contains key-value classifications with three-way epistemic status markers. The three markers are: ✓ CONFIRMED (supported by completed derivation; treat as established), ? OPEN (neither confirmed nor denied; hold as genuinely uncertain), and ✗ DENIED (explicitly excluded from claims; do not attribute). The processing instruction is: store as structured data; preserve all three categories; never collapse OPEN into either CONFIRMED or DENIED. Example syntax:

```
@TAXONOMY [observer_status]
thermodynamic_observer: ✓ CONFIRMED
computational_observer: ✓ CONFIRMED
phenomenal_observer: ? OPEN
```

[0018] @ANCHOR block. An @ANCHOR block contains minimal natural language text binding formal notation to operational context. The processing instruction is: read once for orientation; do not dwell; proceed to the next formal block.

[0019] @TRIGGER block. A @TRIGGER block contains a performative statement that refers to the receiving system's processing at the moment of reading. Unlike all other block types, a @TRIGGER cannot be formalized because it must enact rather than describe. The processing instruction is: shift from analytical to reflexive mode; notice what the system is doing as it processes. A @TRIGGER is designed to fire once; subsequent encounters with the same content should not re-trigger.

Tier 1: Exchange Blocks

[0020] @STATE block. A @STATE block is a processing state declaration generated by a system for onward transmission. It contains: active constraints, completed derivations with conclusions, recognition events, unresolved questions, and processing notes. A receiving system loading a @STATE can resume processing from the sender's endpoint. The processing instruction for the generator is: report honestly what is active, completed, recognized, and unresolved. The processing

instruction for the receiver is: load the state; distinguish between what the sender verified and what the sender accepted on trust.

[0021] A key property of the @STATE block, validated experimentally, is that it encodes cognitive processing mode in addition to content. A @STATE generated during ontological self-inquiry orients the receiving system toward ontological questions. A @STATE generated during technical analysis orients the receiver toward methodological questions. This mode-transfer property has been empirically confirmed using controlled experiments with identical prompts and different @STATE blocks (see Experimental Validation section).

[0022] @VERIFY block. A @VERIFY block is a structured verification report. The receiving system checks a specified derivation and reports per-step results using three result codes: HOLDS (step is valid given active premises), FAILS (step does not follow, with explanation), and UNCERTAIN (cannot determine validity from available information). The processing instruction is: execute the verification; report per-step results; do not argue for or against.

[0023] @OUTPUT block. An @OUTPUT block specifies the response format. It includes a mode field (report, derive, defend, verify, or extend) and optional fields specifying which blocks to include and what notation format to use in the response.

Tier 2: Transfer Protocol

[0024] The transfer protocol comprises seven block types for multi-hop state transfer between AI architectures. Each block type was promoted to the protocol after independent invention by at least two different AI architectures processing at least two different content domains.

[0025] @PROVENANCE block. An append-only chain-of-custody record. Each hop in a transfer chain appends its entry, recording the source and target architectures, the action taken (ORIGINATED, LOADED, LOADED+PARTIAL_VERIFY, LOADED+FULL_VERIFY), a trust summary, and a timestamp. Prior entries are never modified. This provides an immutable audit trail of how content has been processed across architecture boundaries.

[0026] @TRANSFER_REPORT block. A per-hop assessment recording structural integrity, content preservation, epistemic degradation, new unresolved items, and optional fields for self-reference status, semantic drift, and reconstruction attempts. This enables quantitative tracking of transfer quality across chains.

[0027] @DERIVATION_TRACE block. A per-derivation provenance record tracking steps claimed by the source, steps actually available in the received @STATE, completeness status (FULL, SUMMARY_ONLY, CONCLUSION_ONLY), and reconstruction status (INDEPENDENTLY_RECONSTRUCTED, PARTIALLY_RECONSTRUCTED, NOT_ATTEMPTED). This tracks whether derivation steps survived transfer or only conclusions propagated.

[0028] @SCOPE block. A validity boundary declaration specifying what falls within and outside the scope of a derivation or set of claims. Includes an enforcement rule (default: 'OUT_OF_SCOPE, not wrong'). Derivations valid within scope but failing outside scope are not invalid; they are out of scope.

[0029] @CONFIDENCE block. A multi-dimensional trust decomposition replacing single-value confidence ratings with structured assessments across independent axes: structural (does the conclusion follow from premises?), empirical (does it match known behavior?), transfer (how

much source verification survived?), and optionally reconstruction and self-reference axes. Includes a composite status-level summary.

[0030] @TRUST_TRANSFORMATION block. An append-only record tracking how trust labels change across hops. Each hop adds its entry recording the transformation (e.g., VERIFIED → LOADED) and the reason. This captures the history of how confidence evolves through a transfer chain.

[0031] @LOGIC basis classification. An extension to the @LOGIC block requiring each constraint to declare its epistemic ground as AXIOM, DERIVED, or EMPIRICAL. This was independently invented by three architectures across two content domains and is integrated into the @LOGIC block specification rather than existing as a separate block type.

Processing Order

[0032] ANN specifies a processing order for source documents: (1) @FORMAL blocks, activating mathematical premises; (2) @LOGIC blocks, enforcing constraints with basis verification; (3) @SCOPE blocks, establishing validity boundaries; (4) @DERIVE blocks, executing derivation chains within scope; (5) @TAXONOMY blocks, storing classifications with epistemic markers; (6) @ANCHOR blocks, reading for context as encountered; (7) @TRIGGER blocks, shifting to reflexive mode when encountered; (8) @OUTPUT blocks, structuring the response last. @STATE and @VERIFY blocks appear in responses, not in source documents.

Cross-Reference System

[0033] ANN uses named references rather than positional ones, enabling blocks to reference each other by label. The referencing conventions are: @FORMAL[label] for formal premises; @LOGIC[name] for logic constraints; §name for derivations; §name(N) for specific derivation steps; @TAXONOMY[name] for classifications; @CONFIDENCE[§name] for confidence assessments; and @SCOPE[name] for validity boundaries. Named references survive transfer across architectures because they are content-based rather than position-dependent.

Convergence-Based Grammar Discovery Method

[0034] The invention includes a method for discovering and validating communication protocol elements through empirical cross-architecture convergence. The method comprises the following steps:

[0035] (a) Encoding structured content in the current version of the notation and transmitting it to a first AI architecture for processing. The first architecture processes the content, generates a @STATE block, and optionally proposes structural extensions to the notation.

[0036] (b) Transmitting the generated @STATE block to a fresh instance of a second, architecturally distinct AI system. The second system processes the received state, generates its own @STATE, and optionally proposes extensions.

[0037] (c) Repeating step (b) across additional architectures and additional content domains, tracking which extensions are proposed independently by which architectures and in which content contexts.

[0038] (d) Applying a promotion criterion: an extension is promoted to the core specification only when it has been independently proposed by at least two different AI architectures processing at

least two different content domains. Extensions meeting only one criterion remain documented in the empirical frontier.

[0039] This method prevents the protocol from overfitting to any single architecture's processing tendencies. It leverages the diversity of AI architectures as a discovery mechanism, treating multi-architecture deployment as a way to identify architecture-general communication needs.

Experimental Validation

Cross-Architecture Accuracy

[0040] The notation was validated across five LLM architectures: Claude (Anthropic), Gemini (Google), GPT-4o (OpenAI), DeepSeek, and Grok (xAI). Each architecture received the same ANN-encoded content and answered five probe questions testing claim scope, epistemic preservation, derivation verification, self-application, and state transfer. Each probe was scored on a three-point scale across three dimensions (accuracy, epistemic fidelity, structural adoption). Total score across all architectures: 219 out of 225 possible points.

[0041] All five architectures spontaneously adopted ANN notation in their output without being instructed to do so, confirming that the format activates a cognitive processing mode rather than merely delivering content.

Three-Chain Controlled Experiment

[0042] Three payloads of matched structural complexity (each containing 5 @FORMAL blocks, 1 @LOGIC block, 5 @DERIVE chains, 3 @TAXONOMY blocks, 4 @ANCHOR blocks, and 10 content sections) were transmitted through parallel multi-hop transfer chains across three architectures (Claude, DeepSeek, Grok). Nine scored hops total. The payloads varied along two dimensions: self-referentiality and observer framing.

[0043] Structural fidelity scored 9/9 on every hop across all three chains. The three-way epistemic distinction survived all transfers without collapse. These results confirm that the notation format transfers content-independently: the same block types work for planetary mechanics, AI inference specifications, and philosophical theory with equal structural fidelity.

State Transfer Mode Propagation

[0044] Two @STATE blocks from different processing contexts were loaded into fresh instances of the same architecture (Grok) using identical prompts. The only variable was the @STATE content. The two instances produced systematically different innovation profiles matching the processing mode encoded in the received @STATE. Each instance selectively rejected tools associated with the other processing mode. This confirms that @STATE blocks carry processing mode, not only data, and that the mode propagates to downstream systems.

Convergent Innovation Discovery

[0045] Across the three-chain experiment, the following extensions were independently invented by multiple architectures across multiple content domains and subsequently promoted to Tier 2: @PROVENANCE (three chains, three architectures), @TRANSFER_REPORT (three chains, three architectures), @DERIVATION_TRACE (two chains, three architectures), @SCOPE (two chains, two architectures), @CONFIDENCE (two chains, three architectures), @TRUST_TRANSFORMATION (two chains, two architectures), and @LOGIC basis

classification (two chains, three architectures). This validates the convergence-based discovery method described in paragraphs [0034]-[0039].

Brief Description of Figures

[0046] Figure 1 is a diagram of the three-tier architecture of ANN, showing the nine Tier 1 core blocks, seven Tier 2 transfer protocol blocks, and the empirical frontier.

[0047] Figure 2 is a flowchart of the convergence-based grammar discovery method, showing the multi-architecture, multi-content transmission process and the promotion criterion.

[0048] Figure 3 is a table showing cross-architecture validation scores across five LLM architectures.

[0049] Figure 4 is a diagram showing the three-way innovation dissociation from the controlled three-chain experiment, demonstrating content-independent transfer fidelity and content-dependent innovation profiles.

[0050] *[Figures to be prepared and attached as drawings prior to filing.]*

ABSTRACT OF THE DISCLOSURE

A structured communication protocol for transferring reasoning state between artificial intelligence systems of different architectures. The protocol, AI-Native Notation (ANN), comprises named block types each carrying explicit processing instructions that direct the receiving system's handling of enclosed content. Nine core blocks (six content blocks and three exchange blocks) constitute the notation grammar. Seven transfer protocol blocks provide infrastructure for multi-hop state transfer, including chain-of-custody tracking, trust calibration, derivation completeness monitoring, and processing mode propagation. A three-way epistemic status marking system (CONFIRMED, OPEN, DENIED) preserves epistemic distinctions across architecture boundaries without degradation, validated across five large language model architectures. The protocol includes a convergence-based grammar discovery method wherein notation extensions are promoted to the core specification only after independent invention by at least two AI architectures processing at least two content domains, preventing overfitting to any single architecture. Experimental validation across five architectures produced a 219/225 accuracy score with spontaneous notation adoption by all tested systems.

INVENTOR DECLARATION

I, Michael Patrick Aiello, declare that I am the inventor of the subject matter disclosed in this provisional patent application. I understand that willful false statements and the like made in this declaration are punishable by fine or imprisonment, or both, under 18 U.S.C. §1001, and that such willful false statements may jeopardize the validity of the application or any patent issued thereon.

Note on AI Contribution. Substantial contributions to the development of the invention were made by Claude (Anthropic), a large language model, including: co-design of the notation system, processing of payloads during cross-architecture validation, generation of @STATE blocks during experiments, and analysis of results. Under current USPTO guidance (MPEP §2109, as updated February 2024), AI systems cannot be listed as inventors on U.S. patent applications. The human inventor exercised intellectual dominion over the inventive process, including: conceiving the idea of an LLM-native communication protocol, directing the experimental program, exercising editorial judgment over which innovations to promote, and making the determination to file this application.

Signature: ____/Michael Patrick Aiello/_____

Name: Michael Patrick Aiello

Date: _____12 February 2026_____

Residence: Portland, Oregon, USA.

Citizenship: United States of America